AQI-NOW

AI-POWERED AIR QUALITY DASHBOARD

By Disha Girase & Irnitee Patel

PROJECT OVERVIEW

From Data to Real-Time AQI Insights

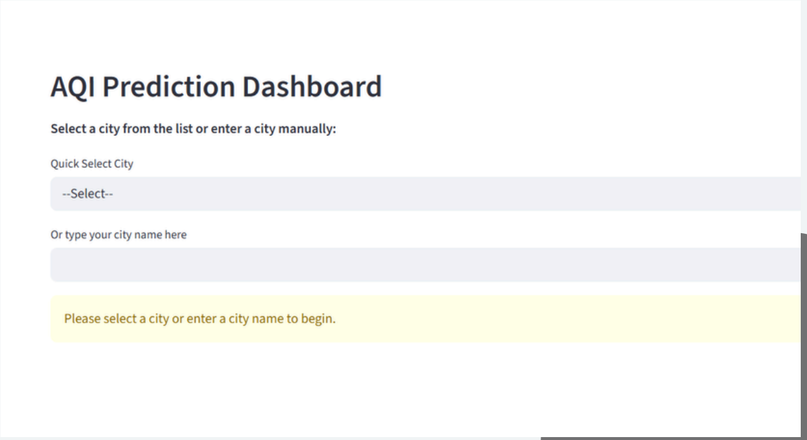
- Predict AQI using Machine Learning
- Combine historical + real-time data
- Interactive web dashboard

```
AQI-Now/
│
├─ app.py
├─ Aqi_model/
│  ├─ aqi_model.joblib
│  ├─ AQI_prediction_using_Machine_Learning.ipynb
│  ├─ city_day.csv
│  └─ test_aqi_model.pyl
├─ requirements.txt
└─ README.md
```

KEY FEATURES

What Makes AQI-Now Powerful?

- Real-time AQI prediction (any city)
- Dropdown + manual input
- Color-coded AQI categories
- Gauge chart + session history



TECH STACK

Category	Technologies Used
Machine Learning	Python, Scikit-learn (Regression Models)
Data Processing	Pandas, NumPy
APIs & Data Sources	OpenWeather API (Air Pollution Data), OpenCage Geocoder
Frontend / Web App	Streamlit
Visualization	Plotly (Gauge Chart)
Model Handling	Joblib (Model Saving & Loading)
Development Tools	Jupyter Notebook, Git, GitHub

DATASET & PREPROCESSING

- **Dataset used:** city_day.csv (Kaggle AQI Dataset)
- **Key features selected:** *PM2.5, PM10, NO_2, CO, AQI*

Removed missing values to ensure data quality

Eliminated duplicate records for consistency

Prepared clean dataset for machine learning models

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	Benzene	Toluene	Xylene	AQI	AQI_Bucket
2	Ahmedabad	01-01-2015			0.92	18.22	17.15		0.92	27.64	133.36	0	0.02	0		
3	Ahmedabad	02-01-2015			0.97	15.69	16.46		0.97	24.55	34.06	3.68	5.5	3.77		
4	Ahmedabad	03-01-2015			17.4	19.3	29.7		17.4	29.07	30.7	6.8	16.4	2.25		
5	Ahmedabad	04-01-2015			1.7	18.48	17.97		1.7	18.59	36.08	4.43	10.14	1		
6	Ahmedabad	05-01-2015			22.1	21.42	37.76		22.1	39.33	39.31	7.01	18.89	2.78		
7	Ahmedabad	06-01-2015			45.41	38.48	81.5		45.41	45.76	46.51	5.42	10.83	1.93		
8	Ahmedabad	07-01-2015			112.16	40.62	130.77		112.16	32.28	33.47	0	0	0		
9	Ahmedabad	08-01-2015			80.87	36.74	96.75		80.87	38.54	31.89	0	0	0		

MACHINE LEARNING MODELS

Training Multiple Regression Models

- Linear Regression
- Random Forest Regressor
- Decision Tree Regressor
- Support Vector Regressor (SVR)
- Gradient Boosting Regressor
- AdaBoost Regressor
- Lasso Regression
- Ridge Regression
- ElasticNet Regression

5. Model Training & Comparison

```
models = [
    ('Linear Regression', LinearRegression()),
    ('Random Forest', RandomForestRegressor()),
    ('Decision Tree', DecisionTreeRegressor()),
    ('SVR', SVR()),
    ('Gradient Boosting', GradientBoostingRegressor()),
    ('AdaBoost', AdaBoostRegressor()),
    ('Lasso', Lasso()),
    ('Ridge', Ridge()),
    ('ElasticNet', ElasticNet())
]
```

MODEL EVALUATION

Selecting the Best Model

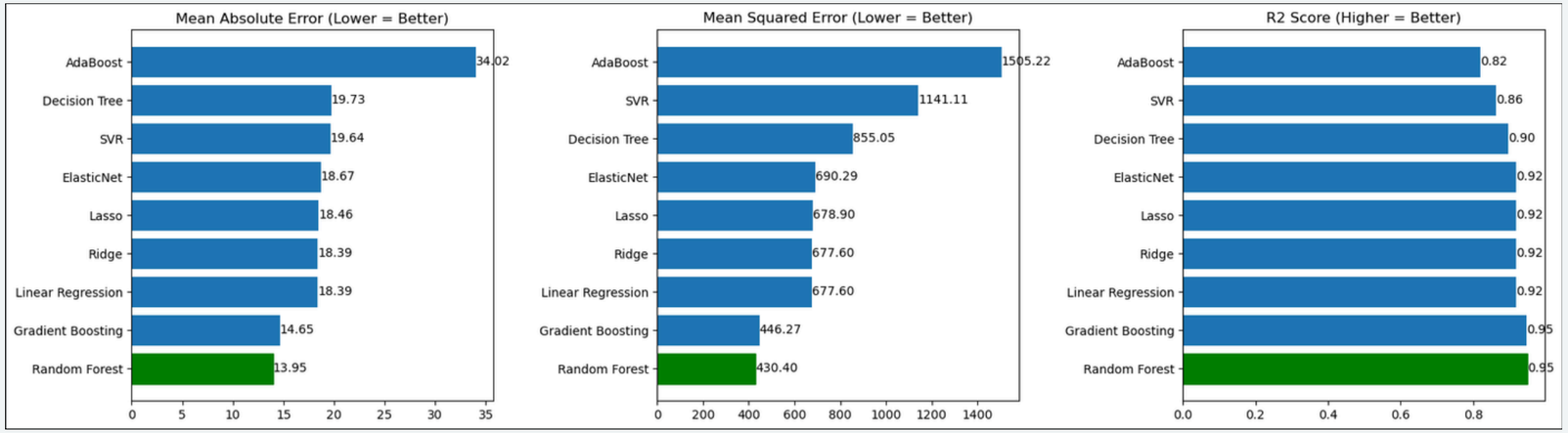
- **Metrics:** MAE, MSE, R^2 Score
- Compared all models.

Random Forest performed best.

```python
results = {'Model': [], 'MAE': [], 'MSE': [], 'R2 Score': []}

for name, model in models:
    model.fit(x_train, y_train)
    y_pred = model.predict(x_test)

    results['Model'].append(name)
    results['MAE'].append(mean_absolute_error(y_test, y_pred))
    results['MSE'].append(mean_squared_error(y_test, y_pred))
    results['R2 Score'].append(r2_score(y_test, y_pred))
```



API INTEGRATION & SYSTEM WORKFLOW

- User enters or selects city name
- **OpenWeather API** fetches real-time pollutant data
- **OpenCage Geocoder** converts city → coordinates
- Pollutant data passed to trained ML model
- Model predicts AQI instantly

```python
API_KEY = "74bd8aa4ee544b2d53c36cf5ef9a93ea"
MODEL_PATH = os.path.join("Aqi_model", "aqi_model.joblib")
```

```python
def fetch_coordinates(city, country=None, api_key=None):
    try:
        query = f"{city},{country}" if country else city
        url = f"http://api.openweathermap.org/geo/1.0/direct?q={query}&limit=1&appid={api_key}"
        res = requests.get(url, timeout=10)
        data = res.json()
        if len(data) == 0:
            return None, None
        return data[0]['lat'], data[0]['lon']
    except Exception:
        return None, None
```

```python
def predict_aqi_with_model(features):

    model_path = os.path.join("Aqi_model", "aqi_model.joblib")

    if not os.path.exists(model_path):
        st.warning(f"ML model '{model_path}' not found. Skipping model prediction.")
        return None
    try:
        model = joblib.load(model_path)
        prediction = model.predict([features])[0]
        return round(float(prediction))
    except Exception as e:
        st.error(f"Error predicting AQI: {e}")
        return None
```

```python
def fetch_aqi_from_api(lat, lon, api_key):
    try:
        url = f"http://api.openweathermap.org/data/2.5/air_pollution?lat={lat}&lon={lon}&appid={api_key}"
        res = requests.get(url, timeout=10)
        data = res.json()
        if "list" not in data or len(data["list"]) == 0:
            return None, None
        aqi = data["list"][0]["main"]["aqi"]
        components = data["list"][0]["components"]
        epa_scale = [50, 100, 150, 200, 300]
        api_aqi = epa_scale[aqi - 1]
        return api_aqi, components
    except Exception:
        return None, None
```

LIVE DEMO & PROJECT ACCESS

- **Step 1:** Open the GitHub repository
- **Step 2:** Read the README for project overview and features
- **Step 3:** Locate the Live Demo link in the README
- **Step 4:** Click the link to launch the dashboard

THANK YOU